

RVISmith: Fuzzing Compilers for RVV Intrinsics

Yibo He
Peking University
Key Lab of HCST (PKU), MOE; SCS
Beijing, China
yibohe@pku.edu.cn

Hongdeng Chen
DAMO Academy, Alibaba Group
Hangzhou, China
hongdeng.chd@alibaba-inc.com

Cunjian Huang
DAMO Academy, Alibaba Group
Hangzhou, China
huangcunjian.huang@alibaba-inc.com

Wei Yang
University of Texas at Dallas
Richardson, USA
wei.yang@utdallas.edu

Xianmiao Qu
DAMO Academy, Alibaba Group
Hangzhou, China
xianmiao.qxm@alibaba-inc.com

Tao Xie*
Peking University
Key Lab of HCST (PKU), MOE; SCS
Beijing, China
taoxie@pku.edu.cn

Abstract

Modern processors are equipped with single instruction multiple data (SIMD) instructions for fine-grained data parallelism. Compiler auto-vectorization techniques that target SIMD instructions face performance limitations due to insufficient information available at compile time, requiring programmers to manually manipulate SIMD instructions. SIMD intrinsics, a type of built-in function provided by modern compilers, enable programmers to manipulate SIMD instructions within high-level programming languages. Bugs in compilers for SIMD intrinsics can introduce potential threats to software security, producing unintended calculation results, data loss, program crashes, etc.

To detect bugs in compilers for SIMD intrinsics, we propose RVISmith, a randomized fuzzer that generates well-defined C programs that include various invocation sequences of RVV (RISC-V Vector Extension) intrinsics. We design RVISmith to achieve the following objectives: (i) achieving high intrinsic coverage, (ii) improving sequence variety, and (iii) without known undefined behaviors. We implement RVISmith based on the ratified RVV intrinsic specification and evaluate our approach with three modern compilers: GCC, LLVM, and XuanTie. Experimental results show that RVISmith achieves 11.5 times higher intrinsic coverage than the state-of-the-art fuzzer for RVV intrinsics. By differential testing that compares results across different compilers, optimizations, and equivalent programs, we detect and report 13 previously unknown bugs of the three compilers under test to date. Of these bugs, 10 are confirmed and another 3 are fixed by the compiler developers.

CCS Concepts

• **Software and its engineering** → **Compilers**; • **Security and privacy** → **Software and application security**.

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '25, Taipei.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1525-9/2025/10
<https://doi.org/10.1145/3719027.3744790>

Keywords

Compiler testing, Fuzzing, RISC-V vector extension, SIMD intrinsics

ACM Reference Format:

Yibo He, Cunjian Huang, Xianmiao Qu, Hongdeng Chen, Wei Yang, and Tao Xie. 2025. RVISmith: Fuzzing Compilers for RVV Intrinsics. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3719027.3744790>

1 Introduction

Modern processors typically support single instruction multiple data (SIMD) instructions, which perform operations on multiple data items in parallel. To use SIMD instructions, programmers have three main approaches: (1) coding assembly instructions, being non-portable, error-prone, and extremely tedious, or (2) compiler auto-vectorization optimizations, or (3) manual vectorization by programming SIMD intrinsics [2, 14, 15]. Although persistent efforts have been made for automatic vectorization [3, 24, 26, 27], compilers still face the inability to apply vectorization and non-optimal optimizations due to limited compile-time information [9, 33]. SIMD intrinsics play a significant role in addressing the preceding limitation of automatic vectorization. As built-in functions inside modern compilers, SIMD intrinsics allow programmers to manipulate SIMD instructions like C functions in high-level programming languages. Given the widespread reliance on SIMD intrinsics for performance-critical software, ensuring the correctness of their compilation is essential [42].

Despite this importance, detecting bugs of compilers for SIMD intrinsics receives little attention in existing research. Previous compiler testing approaches, including both generation-based approaches (e.g., Csmith [43], YARPGen [21, 22]) and mutation-based approaches (e.g., equivalence modulo inputs (EMI) techniques [17, 18, 35]), are unable to generate any programs with SIMD intrinsics. Research work on SIMD-related topics such as automatic vectorization [3, 9, 24, 26, 27, 33] and intrinsic evaluation [30, 34] traditionally focuses on performance but neglects the correctness of compilers for SIMD intrinsics. Compilers for SIMD intrinsics are incorrectly presumed to be inherently robust and resistant to bugs, since most compilations from SIMD intrinsics to SIMD instructions are one-to-one translations. To the best of our knowledge, no prior research work on compiler correctness for SIMD intrinsics has been identified in the research community.

To alleviate the preceding research gap, our work focuses on detecting compiler bugs for RVV (RISC-V Vector Extension) intrinsics. RVV intrinsics are nascent and target the open RISC-V ISA, specifically requiring contributions from the open-source community. RIF (RVV Intrinsic Fuzzing) [32] by SiFive is the only fuzzing tool available for RVV intrinsics that we are aware of, to the best of our knowledge. However, RIF faces inherent limitations, supporting only a restricted subset of intrinsics (less than 7%) and a single operation per strip-mining loop (i.e., a loop that iterates over chunks or strips of data), due to the design to generate accurate calculation results as test oracles. In practical use of RVV intrinsics, e.g., the deep-learning library OpenCV [28], the combination of RVV intrinsics in a loop is much more complex than test cases generated by RIF. A miscompilation bug of LLVM (#106109) related to a specific combination of RVV intrinsics has been reported and cannot be detected by RIF.

In this paper, we propose RVISmith, a randomized fuzzer that generates well-defined C programs that include various invocation sequences of RVV intrinsics. RVISmith addresses the following challenges. **(1) Achieving high intrinsic coverage.** More than 120,000 intrinsics encoded with semantic information and corresponding vector types are defined in the RVV intrinsic document [15], and a random combination of RVV intrinsics is error-prone. We implement RVISmith with a novel technique to generate valid operation sequences in strip-mining loops, supporting more than 98% RVV intrinsics. **(2) Improving the sequence variety.** We introduce vector register allocation and intrinsic scheduling to RVISmith to improve the sequence variety, including the variety of intrinsic combinations and data dependency inside intrinsic sequences. Inspired by the general idea of EMI [17], multiple constraints should be satisfied to ensure the semantic equivalence of different invocation sequences in the same random seed, enriching the test oracle of RVISmith. **(3) Avoiding known undefined behaviors.** Inheriting the unsafe tradition of C, undefined behaviors are ubiquitous in RVV intrinsics. Undefined behaviors in RVV intrinsics exist in different forms from traditional C programs due to various operational semantics, memory-access vectorization, and implementation of inaccessible functions, leading to the failure of existing approaches to detect undefined behaviors, such as clang sanitizer [6] and STACK [40]. Due to the lack of studying undefined behaviors in RVV intrinsics, we struggle with undefined behaviors in the development of RVISmith. By systematically inspecting divergent execution cases and engaging with the RISC-V community, we apply multiple strategies to avoid undefined behaviors to both sequence generation and data generation. We report the detected undefined behaviors of RVV intrinsics as a reference for future work, constituting an additional contribution.

RVISmith generates code with RVV intrinsics in four steps. (1) Preprocessing and sequence selection. Given RVV-intrinsic definitions under test, RVISmith parses the definitions and stores relevant information with object-oriented data structures. RVISmith randomly selects a sequence of operation intrinsics based on a given ratio of SEW (i.e., selected element width in bits) and LMUL (i.e., length multiplier). (2) Data-flow construction. We implement RVISmith with a random algorithm of register allocation that assigns variables to returned values and parameters in the selected operation intrinsics. (3) Intrinsic scheduling. Intrinsic scheduling is to

insert load intrinsics and store intrinsics into the selected operation intrinsics to construct a complete and valid invocation sequence of RVV intrinsics. (4) Code generation. Code generated by RVISmith initializes element values in allocated memory, loads data from memory to vector-type variables, processes constants and data in vector-type variables, stores data from vector-type variables to memory, and prints values of well-defined elements to avoid undefined behaviors and detect bugs. Additionally, RVISmith employs a differential testing framework that compares compilation and execution results across (1) different compilers in a single optimization, (2) a single compiler in different optimizations, and (3) equivalent programs generated by different intrinsic-scheduling algorithms.

To access the effectiveness of RVISmith, we evaluate RVISmith with three modern compilers: GCC, LLVM, and XuanTie. Our experiments show that RVISmith achieves 11.5 times intrinsic coverage higher than RIF, the state-of-the-art fuzzing tool for RVV intrinsics. RVISmith detects 13 previously unknown bugs in the three compilers under test. Among these bugs, 10 are confirmed and another 3 are fixed by the compiler developers. More than 20,000 RVV intrinsics are affected by these bugs. Most of the bugs are miscompilations that are difficult to detect and harmful to software security, leading to unintended calculation results, data loss, and emulator crashes without any compiler warning messages. Moreover, numerous cases generated by RVISmith are found to lead to incorrect results compiled by historical versions of GCC and LLVM (but are correct by the latest version). This finding shows that RVISmith also detects many known compiler bugs. The exact number of known bugs is not reported due to the extensive labor of classification.

In summary, we make the following main contributions:

- **Implementation of fuzzer.** We propose RVISmith that randomly generates well-defined programs with RVV intrinsics. RVISmith is the first tool that can generate complex combinations of RVV intrinsics in strip-mining loops, supporting almost all RVV intrinsics.
- **Detection of real-world bugs.** We detect 13 previously unknown bugs of GCC, LLVM, and XuanTie, improving the reliability and security of mainstream compilers.
- **Empirical study of bugs.** We evaluate RVISmith with multiple historical versions of GCC and LLVM. We report empirical evaluation results showing that compiler bugs related to RVV intrinsics exist widely in versions of GCC and LLVM.
- **Undefined-behavior report.** We make the first report of undefined behaviors caused by RVV intrinsics and how we deal with them in RVISmith. These undefined behaviors lead to typical unsafe issues such as the use of uninitialized variables, and out-of-bound writes. Any future work based on RVV intrinsics can refer to our report.

The implementation of RVISmith based on the ratified RVV intrinsic document in version 1.0 [15] is available at <https://github.com/yibo2000/RVISmith>, and our artifacts are available at <https://zenodo.org/records/15548270>. RVISmith represents a fresh start for both academia and industry in detecting potential compiler bugs related to built-in functions, especially SIMD intrinsics.

2 Background

This section presents some domain knowledge about RVV intrinsics. Two issues are discussed in this section: (1) reasons why modern compilers are equipped with SIMD intrinsics, and (2) a brief introduction to RVV intrinsics.

2.1 Why SIMD Intrinsics?

Domains such as machine learning, image processing, and cloud computing have caused an increase in the significance and sophistication of single instruction multiple data (SIMD) instructions. The key idea of SIMD is, in a single instruction, to calculate multiple data elements simultaneously (e.g., (b) and (c) in Figure 1) rather than data elements one by one (e.g., (a) in Figure 1). Compared to the basic single instruction with single data, SIMD instructions can greatly improve data processing performance. Most modern processors support SIMD instructions.

There are three main approaches for programmers to use SIMD instructions. (1) Embedded assembly or assembly instructions (e.g., Figure 1(c)). Programmers can code an assembly snippet with SIMD instructions, and then embed the assembly snippet to code in high-level programming languages or directly obtain an executable file from the assembly snippet. (2) Compiler optimizations by automatic vectorization (e.g., translation from Figure 1(a) to Figure 1(c)). Modern compilers typically are implemented with auto-vectorization optimizations that generate SIMD instructions from scalar code, including loop-level vectorization [3, 27] and superword-level parallelism (SLP) [5, 24]. (3) Manual vectorization by SIMD intrinsics (e.g., Figure 1(b)). SIMD intrinsics are designed to encapsulate SIMD instructions, allowing programmers to manipulate SIMD instructions like C functions. SIMD intrinsics are typically built-in functions, with functionality implemented by compilers. Compilers release programmers from tedious procedures for using SIMD instructions such as register allocation, and setting control and status registers.

The preceding three approaches for using SIMD instructions each have their own applicable scenarios. Assembly instructions can be used for extremely fine-grained optimization even at clock-cycle level. However, coding assembly instructions is extremely error-prone and tedious, and assembly instructions lack platform portability, resulting in very limited applicable scenarios of coding SIMD assembly instructions for data parallelism. Compiler optimizations by automatic vectorization can improve the performance of data processing. However, the ability of compilers for auto-vectorization depends on the compiler's capability at compile time to analyze a program for precise information [36], and the actual performance after auto-vectorization optimizations is far from the architectural peak performance due to various obstacles, non-optimal optimizations, and inability to obtain information at compile time [9, 33]. Given the preceding limitations of coding assembly and compiler optimizations by automatic vectorization, SIMD intrinsics are widely used in modern processors [2, 14, 15].

2.2 RVV Intrinsics

In this section, we provide a brief introduction to RVV intrinsics. The domain knowledge of RVV intrinsics is integral to the design of RVISmith, as RVV intrinsics are the primary subjects under test. For

a complete introduction, please refer to the ratified RVV intrinsic document [15].

Type system. SIMD instructions are typically used together with a group of vector registers that have larger lengths than regular registers for holding multiple elements during vectorization. The number of bits in a single vector register in RVV is marked as VLEN, which is an implementation-defined constant parameter. SEW (Selected Element Width) is dynamic to determine the size of elements in bits being processed. By default, a vector register is viewed as being divided into VLEN/SEW elements by default. RVV also supports the length multiplier (LMUL), which allows us to process a single vector register (LMUL = 1), multiple vector registers as a vector group (LMUL > 1), and a fraction of a vector register (LMUL < 1).

RVV intrinsics are equipped with an extended type system, encoding information such as SEW and LMUL into vector types shown in Figure 2. For example, `vuint8m2_t` represents that elements of this type use two vector registers as a register group, and each element in the register group is an 8-bit unsigned integer. An exception of type naming in the type system of RVV intrinsics is the bool vector type. As we all know, a bool element requires only one bit, and for this reason, a bool vector type uses only one vector register to optimize resource usage. The first $VLEN \times LMUL \div SEW$ (which is less than VLEN) bits of the bool vector register represent the valid bool elements, and the number in a bool vector type represents the ratio of SEW/LMUL as shown in Figure 2.

Intrinsic naming scheme. The names of RVV intrinsics can generally be divided into four parts as shown in Figure 2. (1) Prefix. In the ratified version of RVV intrinsic document [15], all intrinsic names have an identical prefix `__riscv_` to avoid potential naming conflicts. (2) Mnemonic. A mnemonic is the RVV instruction name after replacing the dots with underscores. For example, the `__riscv_vadd_vv_i8mf8_tumu` intrinsic in Figure 2 uses a mnemonic from the `vadd.vv` instruction, which means adding two signed integer vectors. (3) Vector type. RVV intrinsic names are explicitly encoded with the main vector type used in the calculation, e.g., `i8mf8` in the `__riscv_vadd_vv_i8mf8_tumu` intrinsic in Figure 2. For most RVV intrinsics, the main vector types are unique. A small portion of RVV intrinsics are encoded with more than one vector type such as `__riscv_vreinterpret_v_i8mf8_u8mf8` to avoid naming conflicts. (4) Suffix. A suffix in an intrinsic name defines whether the intrinsic is a masking operation, how to deal with masked-off elements, and how to deal with tail elements. Masked-off elements are those elements that do not need to be operated, and tail elements are elements in unused positions of a vector register. There are two options to deal with masked-off elements and tail elements in RVV: undisturbed (i.e., keeping original values) and agnostic (i.e., unknown values).

There are four types of RVV intrinsic names to avoid always writing long names during using RVV intrinsics. (1) Explicit (non-overloaded) intrinsics. RVV intrinsics in this type do not have policy suffixes that define how to deal with masked-off elements and tail elements. The default option is tail agnostic and masked-off agnostic. (2) Explicit (non-overloaded) intrinsics, policy variants. RVV intrinsics in this type have all four parts in names shown in Figure 2. (3) Implicit (overloaded) intrinsics. Most intrinsics of this type are not encoded with vector types or policy suffixes. (4)

<pre>for(int i=0; i<length; i++){ c[i] = a[i] + b[i]; }</pre>	<pre>for (int avl=length; avl>0; avl-=v1,a+=v1,b+=v1,c+=v1) { v1 = __riscv_vsetvl_e32m2(avl); vfloat32m2_t va = __riscv_vle32_v_f32m2(a, v1); vfloat32m2_t vb = __riscv_vle32_v_f32m2(b, v1); vfloat32m2_t vc = __riscv_vfadd_vv_f32m2(va, vb, v1); __riscv_vse32_v_f32m2(c, vc, v1); }</pre>	<pre>vsetvli zero,a5,e32,m2,ta,ma vle32.v v2,0(a0) vle32.v v4,0(a1) vfadd.vv v2,v2,v4 vse32.v v2,0(a2)</pre>
(a) Scalar code	(b) SIMD intrinsic code	(c) SIMD assembly instructions

Figure 1: Three implementations for adding two 32-bit floating-point arrays. (b) and (c) are for RISC-V V extension.

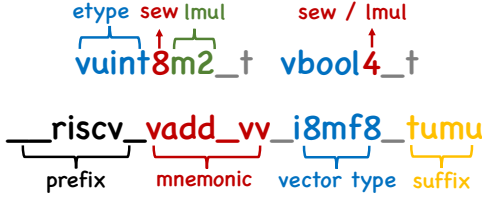


Figure 2: Diagram of vector type naming and intrinsic naming for RVV intrinsics.

Implicit (overloaded) intrinsics, policy variants. Most of intrinsics in this type are not encoded with vector types.

Control and Status Registers (CSRs). Multiple CSRs exist in the RVV programmer model, including seven unprivileged CSRs added by the V extension, CSRs in the base ISA and CSRs in other extensions. CSRs are not directly controlled by intrinsic programmers. Intrinsic programmers can specify or get the status of partial CSRs by calling related intrinsics and setting corresponding arguments. Other CSRs (such as `vstart`) that are not exposed to the intrinsic level are excluded from being controlled at the intrinsic level. Compilers for RVV intrinsics are responsible for generating correct instructions that read and write CSRs.

We provide a brief introduction of intrinsics and parameters related to CSRs. The parameter “unsigned int `frm`” specifies the floating rounding mode CSR (`frm`) for partial floating-point intrinsics. The parameter “unsigned int `vfrm`” specifies the fixed-point rounding mode CSR (`vfrm`) for most fixed-point intrinsics. The `__riscv_vlenb` intrinsic returns the value inside the read-only CSR `vlenb`.

`vtype` and `v1` CSRs are special, which do not require explicit values when intrinsics are programmed. The `vtype` CSR provides how to interpret the contents of vector registers, including SEW, LMUL, whether masked-off elements are agnostic, whether tail elements are agnostic, etc. The `v1` CSR provides an unsigned integer specifying the number of elements to be updated by a vector instruction. Compilers are responsible for controlling the status of `vtype` and `v1` CSRs during translating each RVV intrinsic. At the intrinsic level, the `vtype` CSR is specified by intrinsic names as discussed earlier, and the `v1` CSR is mostly related to the “size_t `v1`” parameter that exists in most RVV intrinsics. Under the programming specification of RVV intrinsics, programmers obtain a `size_t` value by appropriate `vsetvli` or `vsetvlmax` intrinsics, and then use the value as the argument for the “size_t `v1`” parameter, which specifies the number of elements to be updated in each iteration. Formulas

of `vsetvli` and `vsetvlmax` intrinsics are shown as follows:

$$vsetvli(avl) = \min(avl, vlmax) \quad vsetvlmax() = vlmax$$

$$vlmax = VLEN \times LMUL \div SEW$$

In the formula, `avl` is the application vector length, which represents the length of the remaining vector to be processed in the program. `VLMAX` is the maximum number of elements in a vector that one RVV intrinsic can process, given SEW and LMUL. From this formula, we can know that the maximum number of elements in each intrinsic’s iteration is equal if the ratio of corresponding SEW / LMUL is equal.

3 RVISmith

Figure 3 shows an overview of RVISmith’s workflow. Before delving into the technical details, we briefly introduce (1) how RVISmith generates well-defined programs, and (2) how we use the generated programs to detect compiler bugs by differential testing.

Generation of well-defined programs. RVISmith parses the document of RVV intrinsics and selects a ratio-based sequence of RVV intrinsics (Section 3.1). This step ensures that a representative and diverse mix of intrinsics are covered during testing, reflecting realistic usage patterns. Then, RVISmith constructs data dependencies inside the sequence based on a randomized algorithm of vector-register allocation (Section 3.2). This step introduces realistic data dependency chains. Next, RVISmith performs intrinsic scheduling to insert load intrinsics and store intrinsics and constructs multiple equivalent sequences by different scheduling algorithms (Section 3.3). These variants enable broader coverage and robustness in differential testing. Finally, RVISmith generates complete programs, adding code snippets that initialize allocated memory and pointers, update loop variables, and print non-agnostic elements for differential testing (Section 3.4). We also discuss undefined behaviors of RVV intrinsics found by us and how RVISmith avoids these undefined behaviors (Section 3.5).

Differential testing. Three differential-testing strategies are used in our work after generating programs with RVV intrinsics, comparing compilation and execution results from (1) different compilers in the same optimization, (2) the same compiler in different optimizations, and (3) equivalent programs (that are generated by different intrinsic-scheduling algorithms) compiled by the same compiler in the same optimization. Any compiler crashes, runtime crashes, and different execution results indicate detected bug cases (mainly compiler bugs). We manually minimize and classify the detected bug cases by delta debugging, observing error behaviors, and communicating with developers.

The experiments in this work focus on only compiler fuzzing. In theory, RVISmith can also be used to test emulators and CPUs for RVV instructions by differential testing on different hardware, but this topic is beyond the scope of our paper.

3.1 Preprocessing and Sequence Selection

The preprocessing procedure consists of two key aspects given the document of RVV intrinsics under test. (1) RVISmith filters out irrelevant text and parses the given RVV-intrinsic definitions in the document. RVISmith allows users to test any subset of all RVV intrinsics, as some compilers support only partial RVV intrinsics and some processors support only partial RVV instructions. For example, RVV intrinsics involving 64-bit elements are not supported by 32-bit processors. (2) RVISmith divides RVV intrinsics under test into four categories according to intrinsic functionality by a static analysis of definitions. As introduced in Section 2.2, the intrinsic name portion in an RVV-intrinsic definition is encoded with how the intrinsic is used (e.g., corresponding assembly instruction, return type, parameter list, policy), and RVISmith constructs objects to store this information by our object-oriented data structures. The four categories are as follows:

- Load intrinsics: reading data from memory to vector-type variables. Load intrinsics can be recognized by matching mnemonic with RVV load instructions such as `vle{i}.v`.
- Store intrinsics: writing data from vector-type variables to memory. Store intrinsics are void return functions with the common prefix `__riscv_vs` in the design of RVV intrinsics.
- Ignored intrinsics: `vsetvl` intrinsics, `vsetvlmax` intrinsics, and unsupported intrinsics (mainly fault-only-first load intrinsics).
- Operation intrinsics: processing constants and vector-type variables. All intrinsics outside the preceding three categories are classified as operation intrinsics by RVISmith.

After preprocessing, RVISmith randomly selects a ratio-aligned sequence (Definition 3.2) from operation intrinsics under test. This sequence represents the computational operations in a strip-mining loop (i.e., a loop that iterates over chunks of data) in the generated code. The current version of RVISmith generates only one strip-mining loop with multiple RVV intrinsics in each program. Sequence selection in RVISmith is based on a specified ratio of $SEW / LMUL$ (as discussed in Section 2.2, SEW is the size in bits of elements that are being processed, and $LMUL$ is the length multiplier). We provide the definition of ratio-aligned intrinsics and the definition of ratio-aligned intrinsic sequences as follows:

Definition 3.1 (Ratio-aligned intrinsic). An intrinsic with at least one vector type is ratio-aligned, if and only if all vector types in the intrinsic (including return type and parameter type) share the same ratio of $SEW / LMUL$, and the ratio is the common ratio of this intrinsic.

Definition 3.2 (Ratio-aligned intrinsic sequence). A sequence of RVV intrinsics is ratio-aligned, if and only if (1) all ratio-aligned intrinsics in the sequence share the same common ratio, and (2) all intrinsics in the sequence that are not ratio-aligned include at least one vector type with the same ratio as the common ratio of ratio-aligned intrinsics in the sequence.

The reason that sequences selected by RVISmith should be ratio-aligned is as follows. RVISmith seeks to generate VLA-Style (Vector Length Agnostic Style) programs that are portable under different $VLEN$ settings, i.e., the number of bits (in a single vector register) that are decided by processors. A ratio-aligned intrinsic sequence ensures that all elements of an input vector can be processed in the sequence's strip-mining loop without skipped elements, as `vsetvl` and `vsetvlmax` intrinsics return the identical unsigned integer (determining how many elements are processed per iteration) given the same ratio of $SEW / LMUL$ as discussed in Section 2.2. If a sequence of RVV intrinsics is not ratio-aligned and uses `vsetvl` intrinsics as arguments for `vl` parameters and to update loop variables, elements are either processed repeatedly or skipped, and the number of skipped elements depends on the return values of `vsetvl` intrinsics, resulting in a violation of the VLA-Style. For example, as the code in Figure 4, $1/2 vl$ (when $avl > vlmax$) of elements are skipped by `f32m2` intrinsics as the `vl` is for the preceding `e32m4` intrinsics.

To select a ratio-aligned intrinsic sequence, RVISmith extracts the ratio of $SEW / LMUL$ from a user-specified vector type, filters out all intrinsics that can make up a ratio-aligned intrinsic sequence given the ratio, and randomly selects n (specified by users) operation intrinsics from the filtered intrinsics. There are two types of exceptions. (1) Type conversion intrinsics. Type conversion intrinsics always lead to undefined behaviors. We discuss how to deal with these intrinsics in Section 3.5. (2) Reduction intrinsics. During using reduction intrinsics, only the `vs2` parameter (i.e., the second parameter under operation except masking) is iterated in the loop to reduce the vector dimensionality, such as intrinsics for the `vreduor` instruction ($vd[0] = or(vs1[0], vs2[*])$, where $[*]$ denotes all active elements). For reduction intrinsics, RVISmith selects only those whose ratios of types of the `vs2` parameter are the same as the given common ratio.

3.2 Data-Flow Construction

We implement RVISmith with a randomized algorithm of vector-register allocation, which assigns variables to parameters and returned values in the sequence of ratio-aligned operation intrinsics selected by RVISmith, making programs generated by RVISmith (1) follow the use-define chain convention, and (2) cover all four data dependency scenarios (read-read, read-write, write-read, and write-write). Given that the main types of RVV intrinsics are vector types, RVISmith focuses on the allocation of variables in vector types. For scalars and CSRs in parameters, RVISmith randomly generates corresponding constants during code generation.

Given a sequence of selected operation intrinsics, vector-register allocation in RVISmith works as Algorithm 1. RVISmith maintains a key-value table of vector registers during register allocation. Keys of the vector-register table are strings that represent vector types, and values of the vector-register table are arrays of strings that represent active variables in the corresponding vector types. Whenever a new register R in type T is allocated, the vector-register table appends R to the array of T . The function `CoinFlip()` randomly returns either `true` or `false`, which determines whether RVISmith allocates a new register or uses a currently active register. Vector registers newly allocated for parameters of operation intrinsics have a common

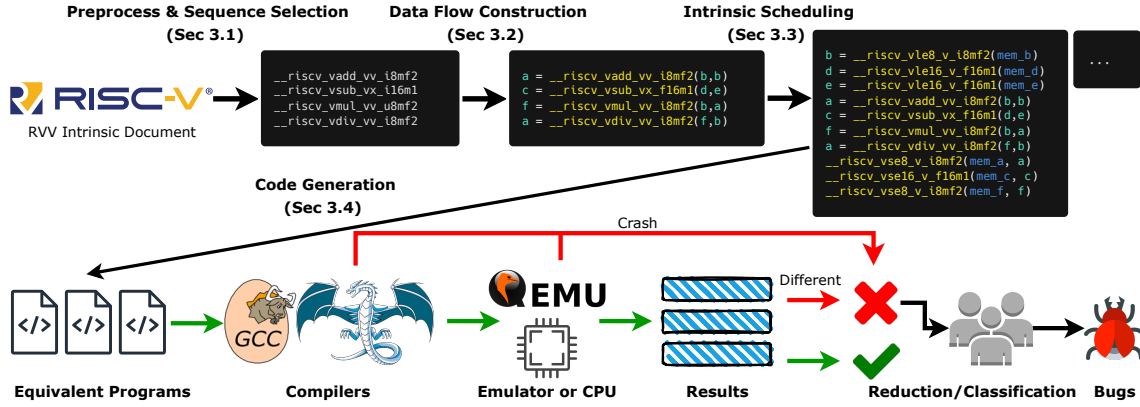


Figure 3: Overview of RVISmith.

```

v1 = __riscv_vsetvl_e32m4(av1);
/* some e32m4 (ratio=8) intrinsics */
vfloat32m2_t va = __riscv_vle32_v_f32m2(a, v1);
vfloat32m2_t vb = __riscv_vle32_v_f32m2(b, v1);
vfloat32m2_t vc = __riscv_vfadd_vv_f32m2(va, vb, v1);
__riscv_vse32_v_f32m2(c, vc, v1); //ratio=16
a += v1; b += v1; c += v1;
av1 -= v1;

```

Figure 4: An example of non-ratio-aligned RVI sequence.

Algorithm 1 Vector-Register Allocation

```

1: procedure VREGISTERALLOCATION(Operation intrinsics I)
2:   VregTable  $\leftarrow \{\}$ 
3:   for all  $op \in I$  do
4:     for all  $p \in op.vector\_parameters$  do
5:       if CoinFlip() or VregTable[p.type] is [ ] then
6:          $\triangleright$  A load intrinsic for vreg_mem is required for
           intrinsic scheduling.
7:         p.vreg  $\leftarrow$  new vreg_mem
8:         VregTable[p.type].append(p.vreg)
9:       else
10:        p.vreg  $\leftarrow$  randomSelect(VregTable[p.type])
11:     for ret  $\leftarrow op.vector\_ret$  do
12:       if CoinFlip() or VregTable[ret.type] is [ ] then
13:          $\triangleright$  No load intrinsic for vreg is required for in-
           trinsic scheduling.
14:         ret.vreg  $\leftarrow$  new vreg
15:         VregTable[ret.type].append(ret.vreg)
16:       else
17:        ret.vreg  $\leftarrow$  randomSelect(VregTable[ret.type])
18:   return I

```

suffix _mem (Line 7), which means that a load intrinsic for this register is required for the following intrinsic scheduling.

3.3 Intrinsic Scheduling

Intrinsic scheduling is to obtain a complete invocation sequence of RRV intrinsics by inserting load intrinsics and store intrinsics into selected operation intrinsics. For each vector variable in parameters of operation intrinsics, if this variable occurs for the first time, a load intrinsic should be inserted before the operation intrinsic. For each vector variable assigned by return values of operation intrinsics, a store intrinsic should be inserted after the operation intrinsic.

We model the problem of intrinsic scheduling as follows. Given a sequence of operation intrinsics I after data flow construction, for each intrinsic $I[i] \in I$, there is an array of prefix intrinsics $P[i]$ (mainly load intrinsics) that should be called before $I[i]$, and an array of suffix intrinsics $S[i]$ (mainly store intrinsics) that should be called after $I[i]$. Let N be length of the sequence of operation intrinsics (i.e., common size), the intrinsic scheduling should satisfy the following constraints for syntactic correctness:

- For each $i \in \text{range}(0, N)$, all intrinsics in $P[i]$ should be executed before $I[i]$, and all intrinsics in $S[i]$ should be executed after $I[i]$.
- For any $i, j \in \text{range}(0, N)$, if $i < j$, $I[i]$ should be executed before $I[j]$.
- For any $x \in \text{range}(0, N)$, for any $i, j \in \text{range}(0, P[x].size)$, if $i < j$, $P[x][i]$ should be executed before $P[x][j]$.
- For any $x \in \text{range}(0, N)$, for any $i, j \in \text{range}(0, S[x].size)$, if $i < j$, $S[x][i]$ should be executed before $S[x][j]$.

To satisfy the preceding constraints, we implement RVISmith with three intrinsic-scheduling algorithms. We provide the intrinsic-scheduling algorithms in Algorithms 2, 3, and 4. (1) **All-in intrinsic scheduling** (in Algorithms 2). All prefix intrinsics are called at the beginning of the sequence (Lines 3-5). Operation intrinsics are called between prefix intrinsics and suffix intrinsics (Lines 6-7). All suffix intrinsics are called at the end of the sequence (Lines 8-10). (2) **Unit intrinsic scheduling** (in Algorithms 3). Prefix intrinsics and suffix intrinsics are called immediately before/after the corresponding operation intrinsic (Lines 3-8). (3) **Random intrinsic scheduling** (in Algorithms 4). Intrinsics can be called at any location that satisfies the preceding constraints (Lines 4-11). The function randomInsert

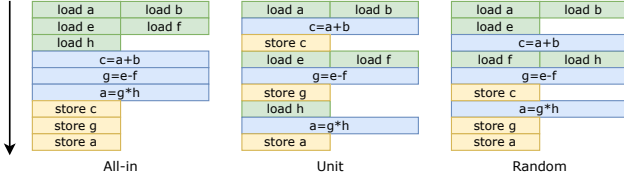


Figure 5: An example of invocation sequences after intrinsic scheduling.

Algorithm 2 All-in Intrinsic Scheduling

Require: Prefix intrinsics P , suffix intrinsics S , operation intrinsic I , common size N .

```

1: function SCHEDULING_ALLIN( $P, S, I, N$ )
2:    $res \leftarrow []$ 
3:   for  $i \leftarrow 0$  to  $N - 1$  do
4:     for all  $p \in P[i]$  do
5:        $res.push\_back(p)$ 
6:     for  $i \leftarrow 0$  to  $N - 1$  do
7:        $res.push\_back(I[i])$ 
8:     for  $i \leftarrow 0$  to  $N - 1$  do
9:       for all  $s \in S[i]$  do
10:         $res.push\_back(s)$ 
11:   return  $res$ 

```

Algorithm 3 Unit Intrinsic Scheduling

Require: Prefix intrinsics P , suffix intrinsics S , operation intrinsic I , common size N .

```

1: function SCHEDULING_UNIT( $P, S, I, N$ )
2:    $res \leftarrow []$ 
3:   for  $i \leftarrow 0$  to  $N - 1$  do
4:     for all  $p \in P[i]$  do
5:        $res.push\_back(p)$ 
6:      $res.push\_back(I[i])$ 
7:     for all  $s \in S[i]$  do
8:        $res.push\_back(s)$ 
9:   return  $res$ 

```

randomly inserts a value between the begin pointer and the end pointer and returns the pointer to the inserted value.

Figure 5 shows an example of invocation sequences after intrinsic scheduling for a more precise presentation. The all-in algorithm and the unit algorithm represent extreme cases: prefix/suffix intrinsics are called at the beginning/end, or adjacent to the corresponding operation intrinsics. The random algorithm represents general cases: all well-defined invocation sequences can occur.

Equivalent programs. Programs generated by RVISmith with different scheduling algorithms are equivalent in semantics. The basic idea is that positions of load intrinsics and store intrinsics during scheduling should not change the final values of elements. RVISmith implements this idea by simply separating the memory allocated for load intrinsics and store intrinsics during code generation. This implementation ensures that memory is not modified by store intrinsics before the memory is accessed by load intrinsics.

Algorithm 4 Random Intrinsic Scheduling

Require: Prefix intrinsics P , suffix intrinsics S , operation intrinsic I , common size N .

```

1: function SCHEDULING_RANDOM( $P, S, I, N$ )
2:    $res \leftarrow []$ 
3:    $ptr\_op \leftarrow res.begin()$  point to the last inserted op intrinsic
4:   for  $i \leftarrow 0$  to  $N - 1$  do
5:      $ptr\_op \leftarrow randomInsert(ptr\_op, res.end(), I[i])$ 
6:      $ptr\_begin \leftarrow res.begin(), ptr\_end \leftarrow ptr\_op$ 
7:     for all  $p \in P[i]$  do
8:        $ptr\_begin \leftarrow randomInsert(ptr\_begin, ptr\_end, p)$ 
9:      $ptr\_begin \leftarrow ptr\_op, ptr\_end \leftarrow res.end()$ 
10:    for all  $s \in S[i]$  do
11:       $ptr\_begin \leftarrow randomInsert(ptr\_begin, ptr\_end, s)$ 
12:   return  $res$ 

```

Table 1: Scalar data generation in RVISmith.

Type	Bits	Generation range (n bits)
bool	1	[0, 1]
int	8, 16, 32, 64	$[-2^{n-1}, 2^{n-1} - 1]$
uint	8, 16, 32, 64	$[0, 2^n - 1]$
float	8, 16, 32, 64	$uint2float_binary(uint(n))^*$

* If the return value is NaN, generate zero.

3.4 Code Generation

After a complete invocation sequence of RVV intrinsics in the preceding steps, there are four steps left to get a valid C program with RVV intrinsics. (1) Global variable declaration. RVISmith uses a series of global variables as the allocated memory to be processed. Each vector register `vreg_mem` (in Algorithm 1, Line 7) corresponds to a global variable as well as a load intrinsic. (2) Memory initialization. Before operating RVV intrinsics, all global variables are initialized by randomly generated values of the corresponding type. To avoid potential floating-point problems such as floating-point precision, RVISmith generates an unsigned integer value in corresponding bits and converts the value to a float by a union data structure. How RVISmith initializes scalars in memory is shown in Table 1. (3) Loop generation. Statements that initialize and update loop variables are added. (4) Memory print. After all operations are finished, programs generated by RVISmith output the final values of all non-agnostic elements in memory for differential testing. We discuss how to judge whether an element is non-agnostic in the Section 3.5.

3.5 Avoiding Undefined Behaviors

General undefined behaviors of C programs are discussed in previous work, such as Csmith [43]. In this section, we focus on the undefined behaviors that are related to RVV intrinsics. RVISmith avoids all the following undefined behaviors.

Masked-off elements and tail elements. RVV intrinsics use the masking mechanism to represent whether an element should be executed (i.e., an implementation of control flow). Masked-off elements are those with a zero mask, indicating that the elements

do not need to be executed. Tail elements refer to the unused elements in vector registers. For example, if a vector register can hold eight elements but only six are used, the remaining two elements are considered tail elements. For mask-agnostic intrinsics and tail-agnostic intrinsics, values of masked-off elements and tail elements are unknown (i.e., non-agnostic). RVISmith is implemented with an agnostic-state model to label active elements (i.e., elements neither masked-off nor tail) and generates print statements for only active elements. In the agnostic-state model, an element is non-agnostic if and only if this element is active and all source elements are non-agnostic. RVISmith ensures that every printed element is well defined.

Conditionally undefined intrinsics. Some RVV intrinsics conditionally return uninitialized values for active elements. `vrgather` intrinsics, `vsld` intrinsics, `vcompress` intrinsics, `vcpop` intrinsics, `vfirst` intrinsics, `vmisf` intrinsics, `vmisb` intrinsics, `vmisof` intrinsics, and `viota` intrinsics are all conditionally undefined intrinsics as found by us. For intrinsics of this type, RVISmith uses a rule-based technique to generate “absolutely correct” values instead of random values in Table 1 as arguments to ensure that the return values are well defined (i.e., no uninitialized values). The specific rule of data generation depends on the semantic information of each intrinsic.

Intrinsics that are always undefined. Some RVV intrinsics always return uninitialized values for active elements. All values returned by `vundefined` intrinsics are uninitialized. The values in the extended portion from `vlmul_ext` intrinsics are uninitialized. It is difficult to determine whether the values returned by `vreinterpret` intrinsics are well defined. For intrinsics of this type, RVISmith removes the returned vector register from the `VregTable` during data flow construction to prevent contamination of subsequent elements. RVISmith also does not generate print statements for values returned by these intrinsics during code generation.

Array safety. To avoid the problem of array index out-of-bound, programs generated by RVISmith adhere to the programming specifications of RVV intrinsics. Arguments of “size vl” parameters are returned from `vsetvl` intrinsics. Arguments of index parameters in indexed load/store intrinsics are returned from `vid` intrinsics. Fine-grained adjustments are applied to the arguments to ensure array safety. Compilers for RVV intrinsics do not perform bound checking, which is known as the correctness-security gap [7]. This undefined behavior can lead to security problems such as out-of-bound write. We provide an out-of-bound write case involving RVV intrinsics; this case is found and fixed during the development of RVISmith and reported in #117677. This out-of-bound write is caused by the absence of adjustments for the vl arguments of segment load/store intrinsics. After continuous communication with RISC-V officials and improvements to the program generation approach used in RVISmith, no out-of-bound arrays currently exist in the programs generated by RVISmith.

Numerical safety. The data generation approach of RVISmith ensures that the initial values of elements are valid and safe. Integer overflow and NaN (Not-a-Number) values may occur during computation by RVV intrinsics. RVISmith converts NaN to a specific valid value only during data generation and print-statement generation. The behavior of integer overflow in RVV intrinsics is well defined by the documentation, and no additional operations are

Table 2: Mainstream compilers for RVV intrinsics.

RVV Spec	GCC	LLVM	XuanTie
Draft (≤ 0.11)	13	16	gcc-v2
Ratified ($0.12\&1.0$)	14, 15*	17, 18, 19, 20*	gcc-v3, llvm-v2

* Experimental version.

required. Elements are used for only computation to avoid causing follow-up issues (e.g., infinite loop). RVISmith does not ensure that every element is valid during computation but ensures that invalid elements do not cause issues or inconsistencies in printed values.

4 Evaluation

This section presents the details of our evaluation of RVISmith. The effectiveness of RVISmith is mainly evaluated from three dimensions: bug detection, coverage (including code coverage and intrinsic coverage), and performance.

4.1 Experimental Setup

Compilers under test. Mainstream compilers for RVV intrinsics are shown in Table 2. Our experiments are limited to compilers that support the ratified RVV intrinsics. GCC ($\geq 14.1.0$) [10] and LLVM ($\geq 17.0.1$) [23], the most popular open-source compilers, both support ratified RVV intrinsics. XuanTie [1] is a compiler developed by Alibaba DAMO Academy based on GCC and LLVM to match XuanTie CPUs (e.g., C906), and also support ratified RVV intrinsics. Compilers that implement only draft versions of RVV intrinsics shown in Table 2 or lack the support of RVV intrinsics (including old versions of GCC, LLVM, and XuanTie) are excluded from our experiments. XuanTie is excluded from code coverage evaluation, as its source code is not available.

Compiler flags. We use all five standard optimization flags, i.e., `-O0`, `-O1`, `-O2`, `-O3`, and `-Os`, for fuzzing compilers in the latest released versions and experimental versions. In the experiments involving historical versions and performance analysis, only the `-O0` and `-O3` optimization flags are used. Other related compiler flags are set as follows: “`-march=rv64gcv_zvfh -mabi=lp64d -Wno-psabi -static`”.

Environment. We conduct all our evaluations on a docker container running Ubuntu 24.04.1 LTS in a Linux server. The Linux server is equipped with two AMD EPYC 7H12 64-Core CPUs and each CPU has 512GB RAM. RISC-V ELF files after compilation are executed by QEMU in version 9.1.0.

4.2 Quantitative Bug-Finding Results

This subsection presents various summary statistics on the results of our compiler bug detection effort.

Number of bugs. Table 3 summarizes 13 previously unknown bugs uncovered by RVISmith in the latest released versions and experimental versions of compilers under test. All the bugs are confirmed as real-world bugs by the corresponding compiler developers, and three bugs are fixed. Six unrepaired bugs of GCC are labeled target milestones, which define when the bugs are expected to be fixed. RVISmith detects only one bug in the latest version (19.1.4 by the time that we experiment) of LLVM, and this bug is

Table 3: Type and status of new compiler bugs.

Symptom	GCC	LLVM	XuanTie	Total
Compiler Crash	2	0	1	3
Runtime Crash	2	1	2	5
Wrong Result	3	0	2	5
Total	7	1	5	13
(Confirmed Fixed)	(6 1)	(0 1)	(4 1)	(10 3)

detected independently by the LLVM developers nearly simultaneously. Five bugs are detected in the latest version of XuanTie, and we report these bugs by emailing XuanTie developers. Three bugs lead to compiler crashes when parsing RVV intrinsics (two null-pointer exceptions and one floating-point exception). Five bugs lead to runtime crashes by generating illegal instructions or trying to access unavailable memory. Five bugs lead to incorrect calculation results.

Comparison with baselines. To demonstrate RVISmith’s superiority in bug detection, we compare its results with two baselines, RIF and Csmith, by measuring the number of detected bugs. During a one-week experiment to fuzz the latest versions of GCC (14.2.0), LLVM (19.1.4), and XuanTie (gcc-v3, llvm-v2), the baseline tools (RIF and Csmith) do not detect any bug, while RVISmith identifies multiple bug cases. As the only tool that supports RVV intrinsics before RVISmith, RIF cannot detect bugs for two reasons. First, RIF is implemented based on the draft specification rather than the ratified specification, leading to numerous compilation errors when compiling generated programs as the RVV specification is updated. Second, as discussed in Section 1, RIF supports extremely limited RVV intrinsics and does not support operation combinations in a loop, resulting in an inherent limitation in bug detection.

Security impacts. Using the programs generated by RVISmith and our differential testing framework, RVISmith can detect not only compiler crashes but also security bugs. The impacts of these security bugs include unintended rounding (e.g., #118103), data loss (e.g., #117947), illegal memory access (e.g., #118100), etc. These bugs are difficult to detect because they occur under seemingly normal conditions, without any compilation warning/error messages. These bugs can lead to serious security issues when bug-triggering SIMD intrinsics are used to accelerate the processing of sensitive data, such as quantitative computations in financial systems.

Affected RVV intrinsics. Some bugs detected by RVISmith are related to specific intrinsics. By manually debugging bug cases, we identify the affected RVV intrinsics of these bugs. Other bugs are caused by overly complex combinations of RVV intrinsics, making it difficult for us to identify the specific affected RVV intrinsics. Table 4 summarizes the known affected RVV intrinsics. Most of non-compiler-crash bugs are caused by faults related to CSRs, especially CSRs that are explicitly controlled by corresponding parameters in intrinsics (e.g., vxrm and frm). Although the number of unique bugs is limited, a significant number of RVV intrinsics are affected by these bugs and have the potential to trigger them. More than 20,000 RVV intrinsics are known to be affected by new bugs detected by RVISmith. The significant number of affected RVV intrinsics reflects the importance of the detected bugs.

Table 4: Number of known affected RVV intrinsics.

Category	Symptom	Compiler	#N
LMUL Extension Intrinsics	Compiler Crash	GCC, XuanTie	270
LMUL Truncation Intrinsics	Compiler Crash	GCC, XuanTie	270
Vector Insertion Intrinsics	Compiler Crash	GCC	292
Fixed-Point Intrinsics with ‘vxrm’	Wrong Result	GCC	4416
Masked Widening Intrinsics	Runtime Crash	XuanTie	7376
Floating-Point Intrinsics with ‘frm’	Runtime Crash Wrong Result	LLVM GCC, XuanTie	9020

Quantitative comparison of GCC and LLVM versions. Figure 6 shows the results of our quantitative comparison experiments. For each major version of GCC and LLVM shown in Table 2, we include the earliest and latest available versions. As GCC has limited versions that support ratified RVV intrinsics, we also include all released versions of GCC (14.[1-2].0). For each compiler under test, we use RVISmith to randomly generate 1,500,000 programs (500,000 seeds * 3 scheduling algorithms), and each program is compiled at -O0 and -O3. The data length is randomly selected from [1, 1000], and the sequence length is randomly selected from [1, 100].

By the quantitative comparison of GCC and LLVM versions, we derive the following findings. First, compiler bugs related to RVV intrinsics exist widely in GCC and LLVM. Among the tested versions, there is no detected bugs for LLVM-20-trunk, as the only detected LLVM bug #117909 illustrated in Figure 11 has been fixed. Second, RVISmith is also capable of detecting many fixed bugs that are not included in Table 3. For example, a large number of programs compiled by LLVM-17 and LLVM-18 lead to different results between -O0 and -O3; however, this issue does not occur with LLVM-19 and LLVM-20. Third, programs that are capable of triggering the same bug are diverse. This finding indicates that a substantial number of intrinsics are affected by the bugs, being consistent with our previous discussion.

We provide explanations for some details in Figure 6. Figure 6(b) does not include the “Compiler Crash” column because RVISmith does not detect any compiler crash in LLVM. A large number of programs fail to compile with LLVM-17 because it does not support certain ratified RVV intrinsics.

Findings on differential-testing strategies. As discussed at the beginning of Section 3, three differential-testing strategies are used, comparing the results from (1) different compilers, (2) different optimizations, and (3) equivalent programs. In our experiments, all previously unknown bugs can be detected by the cross-compiler and cross-optimization strategies. However, we find that comparing results from equivalent programs compiled by a single compiler with a single optimization detects only four previously unknown bugs, and all these bugs are compiler crashes and runtime crashes. This finding provides us with three key insights. First, differential testing between compilers and optimizations is crucial. Most security bugs and logical bugs, rather than crashes, are related to the optimization components in compilers. Second, increasing diversity in invocation sequences by introducing various intrinsic-scheduling algorithms is effective. We observe that some compiler/runtime crashes are triggered by only specific scheduling algorithms. For example, the bug in #117909 cannot be triggered by the unit scheduling

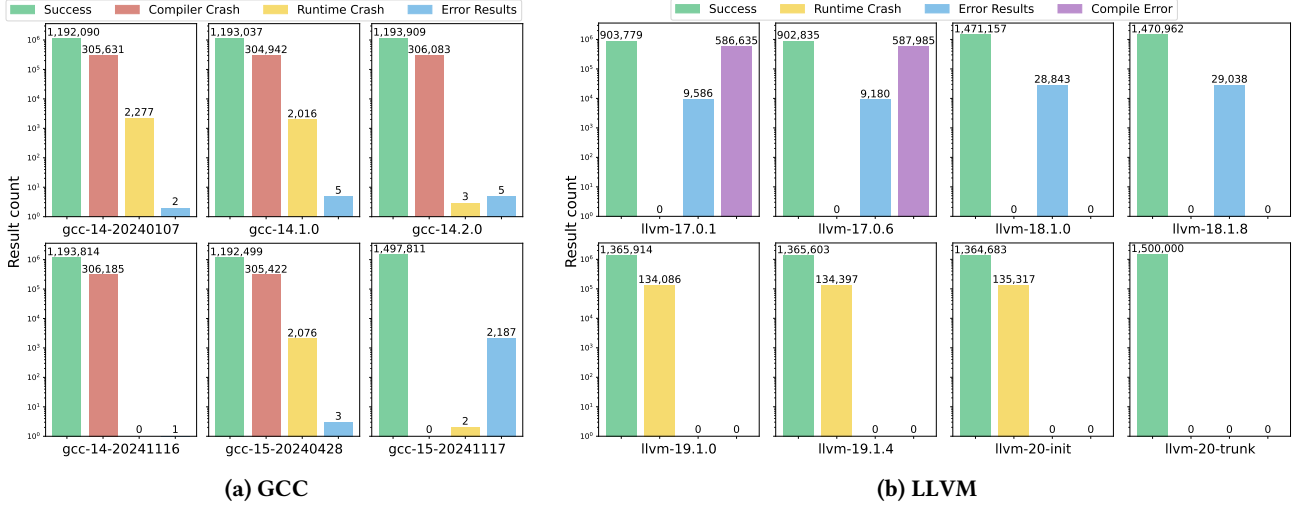


Figure 6: Quantitative comparison results of GCC and LLVM versions.

algorithm. Third, the current construction of equivalent programs is not critical. We do not detect more new bugs by the current equivalence, and more effective approaches for the construction of equivalent programs may help to address this limitation.

4.3 Coverage

Code coverage. We conduct code coverage analysis to evaluate the effectiveness of RVISmith in improving code coverage. We use RVISmith to generate 10,000 random programs (10,000 random seeds * 1 scheduling mode) including all four categories of RVV intrinsics. For comparison against RVISmith, we also use Csmith (a well-known code generator for compiler testing) and RIF (the only compiler fuzzer for RVV intrinsics) as baselines. We use Csmith and RIF to generate 10,000 random programs respectively. For the sake of fairness, the data lengths and operation lengths of RIF and RVISmith are set to 10. We use the -O3 optimization flag when compiling programs generated by Csmith, RIF, and RVISmith. We use gcov for collecting GCC code coverage and llvm-cov for collecting LLVM code coverage. We control the number of test cases rather than the duration of fuzzing, as the generation time proportion of code generators is small and the most time-consuming parts are compilation and execution, which are proportional to the number of test cases (we discuss the performance details in Section 4.4). Based on previous experiments in compiler testing [19, 43], compiling 10,000 random programs that are generated by each generator is sufficient to achieve meaningful code coverage results.

Code coverage results of GCC and LLVM source code are shown in Table 5, and we have two main findings. First, RVISmith achieves significantly higher code coverage than using only existing code generators Csmith and RIF. For example, compared to compiling 10,000 programs generated by Csmith and 10,000 programs generated by RIF, compiling other 10,000 programs generated by RVISmith can remarkably increase line coverage in GCC by 11.81% (124,948 more lines) and in LLVM by 5.78% (91,987 more lines). Second, employing these generators in a complementary manner achieves more code coverage than relying on a single generator

Table 5: Function coverage (FC), line coverage (LC), and branch coverage (BC) of GCC and LLVM source code.

Generator		FC	LC	BC
GCC	RVISmith	26.88%	23.77%	14.90%
	Csmith	14.42%	12.08%	6.25%
	Csmith+RVISmith	27.13%	24.37%	15.21%
	(absolute change)	+16,011	+130,202	+147,470
	RIF	6.05%	6.18%	3.69%
	RIF+RVISmith	27.51%	24.39%	15.20%
	(absolute change)	+27,061	+192,924	+189,344
	Csmith+RIF	15.74%	12.95%	6.64%
LLVM	Csmith+RIF+RVISmith	27.63%	24.76%	15.39%
	(absolute change)	+14,985	+124,948	+143,891
	RVISmith	21.51%	14.46%	13.64%
	Csmith	16.00%	8.82%	6.79%
	Csmith+RVISmith	21.98%	14.91%	14.10%
	(absolute change)	+7,686	+96,872	+70,114
	RIF	8.76%	4.75%	5.05%
	RIF+RVISmith	22.29%	15.05%	14.26%
	(absolute change)	+17,398	+164,001	+88,336
	Csmith+RIF	16.90%	9.50%	8.20%
	Csmith+RIF+RVISmith	22.49%	15.28%	14.48%
	(absolute change)	+7,196	+91,987	+60,221

alone. This finding indicates that each generator uniquely covers some specific portions of the GCC and LLVM source code.

Intrinsic coverage. RVISmith focuses on compilers for RVV intrinsics. However, code coverage of the entire source code may not reflect the extent of testing relevant compiler portions that support RVV intrinsics. GCC and LLVM are huge code projects that contain a substantial amount of complex code unrelated to RVV intrinsics, as the two compilers support multiple source languages, backends, and configuration options.

To address the aforementioned limitation of code coverage, we introduce the intrinsic coverage metric to measure the effectiveness

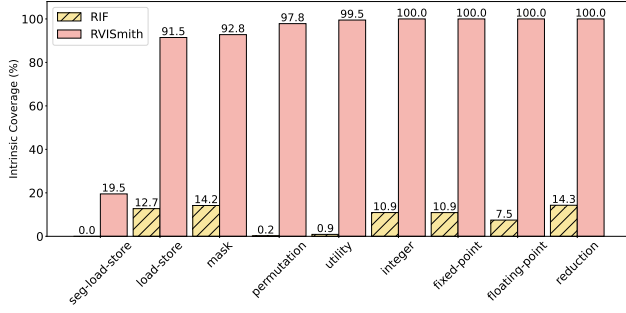


Figure 7: Distribution of covered RVV intrinsics by RIF and RVISmith ($n = 10^5$).

of a code generator in covering RVV intrinsics. Intrinsic coverage provides insights into how thoroughly RVV intrinsics are being tested, helping identify untested parts of RVV intrinsics. How we calculate intrinsic coverage is shown in the following formula. In the formula, $count_i$ represents the count of the appearances of i -th intrinsic's name in the generated code, and $weight_i$ represents the count of the i -th intrinsic's name in the list of intrinsic definitions. Note that $weight_i = 1$ for non-overloaded intrinsics, and $weight_i > 1$ for overloaded intrinsics. By this formula, we can obtain approximate intrinsic coverage with simple static analysis of intrinsics names instead of developing a complex tool to find out which overloaded intrinsic is invoked in a program. This approximation is intentionally designed as overloaded intrinsics (the same SEW/LMUL ratio and the same operation) have an equal probability of coverage by RVISmith.

$$\text{Intrinsic coverage} = \frac{\sum_{i=0}^n \min(count_i, weight_i)}{\sum_{i=0}^n weight_i}$$

We use RIF and RVISmith to generate random programs with n random seeds, where the data lengths and operation lengths are set to 10. Table 6 summarizes the experimental results of RIF and RVISmith in the intrinsic coverage metric. Csmith is excluded from this experiment because it cannot generate any RVV intrinsics. The results show that RVISmith achieves significantly higher intrinsic coverage than RIF across all four categories of RVV intrinsics being tested. RVISmith supports all four categories of RVV intrinsics; however, RIF supports only explicit intrinsics (with and without policy). There is little difference in coverage by RVISmith between explicit (non-overloaded) and implicit (overloaded) intrinsics when test cases are enough ($n \geq 10^4$). Compared to RIF, RVISmith totally achieves 11.5 times higher intrinsic coverage (from 6.39% to 74.08% when n is 10^5).

Figure 7 shows the distribution of RVV intrinsics covered by RIF and RVISmith, with RVV intrinsics classified by functionality according to the RVV intrinsic document. Compared to RVISmith, the intrinsic coverage of RIF is low for any category of RVV intrinsics. RVISmith achieves over 90% intrinsic coverage on all RVV intrinsics except segment load/store intrinsics. We provide the reasons why RVISmith achieves low coverage on segment load/store intrinsics in the experiments. First, the number of segment load/store intrinsics is large. The number of segment load/store intrinsics exceeds

37,000, making it the most numerous among all types. Second, the RVISmith approach has a low probability of generating programs including segment load/store intrinsics, as the number of operation intrinsics capable of reading/writing data via segment load/store intrinsics is much smaller than the total number of segment load/store intrinsics. This low probability results in limited segment load/store intrinsics being inserted during intrinsic scheduling. A coverage-guided fuzzing approach for RVV intrinsic compilers can be explored in future work to address this limitation of RVISmith.

4.4 Performance of RVISmith

We measure the distribution of CPU time and real time used by each main step in our pipeline when fuzzing on the latest released versions of GCC (14.2.0) and LLVM (19.1.4). CPU time is collected by `time.process_time()`, and real time is collected by `time.time()`. RVISmith is used in its default configuration for explicit intrinsics. Our hardware configuration is discussed in Section 4.1. We use `ProcessPoolExecutor` for parallel processing, and all CPU cores are used. As shown in the results in Table 7, the time for RVISmith to generate test cases accounts for a small proportion of both CPU time (0.021%) and real time (0.022%). Most of the time is spent on compilation and execution. This finding indicates that the performance of RVISmith is not a bottleneck when fuzzing GCC and LLVM. The performance analysis also reveals other interesting findings. For example, the proportion of CPU time for GCC and LLVM is similar, but GCC takes approximately twice as much real time as LLVM when compiling test cases.

4.5 Case Study

RVISmith is capable of detecting compiler bugs related to RVV intrinsics. We discuss a selection of bugs discovered by RVISmith.

Data loss. Figure 8 shows a program that triggers a miscompilation bug of GCC. In each iteration, two vectors with `v1` elements each are added by the `vadd_vv` intrinsic. However, when the program is compiled by GCC at `-O2/-O3` optimizations, pointers to the allocated memory are updated by the `CSR vlenb` that is incorrectly configured by a `vsetvli` instruction. This miscompilation causes the `v1` elements to be successfully added, while the next `v1` elements are skipped in each iteration. Finally, half of the data are lost in the computation result.

Unintended rounding. Figure 9 shows a program that triggers an unintended rounding bug of GCC. The first floating-point intrinsic (`vfnmadd_vv`) is in the RDN (Round Down) mode, and the second floating-point intrinsic (`vfmsac_vv`) is in the default RNE (Round to Nearest) mode. In the correctly compiled assembly code, the status of `frm` should be changed before `vfnmadd_vv` and be restored between `vfnmadd_vv` and `vfmsac_vv`. However, GCC misses the instruction that restores the status of `frm`, leading to incorrect calculation results. In certain instances of this bug, the disparity between correct and incorrect results is substantial, as a small number of bits in a floating-point number can significantly affect the numerical value.

Compiler crash. A compiler crash (internal compiler error) is always viewed as a compiler bug because compilers should either work successfully or provide warning/error messages for incorrect input programs. GCC crashes on the program shown in Figure 10.

Table 6: Intrinsic coverage by RIF and RVISmith.

Intrinsic under test	$n = 10^2$		$n = 10^3$		$n = 10^4$		$n = 10^5$	
	RIF	RVISmith	RIF	RVISmith	RIF	RVISmith	RIF	RVISmith
Explicit	5.11%	6.90% (+1.79%)	14.21%	32.09% (+17.88%)	15.18%	65.25% (+50.07%)	15.18%	78.69% (+63.51%)
Explicit, policy	0.75%	4.61% (+3.86%)	5.62%	28.22% (+22.60%)	10.59%	69.92% (+59.33%)	10.60%	70.79% (+60.19%)
Implicit	0.00%	7.27% (+7.27%)	0.00%	42.14% (+42.14%)	0.00%	66.50% (+66.50%)	0.00%	78.65% (+78.65%)
Implicit, policy	0.00%	4.85% (+4.85%)	0.00%	34.77% (+34.77%)	0.00%	70.44% (+70.44%)	0.00%	70.44% (+70.44%)
Total	1.35%	5.74% (+4.39%)	4.76%	33.84% (+29.08%)	6.39%	68.32% (+61.93%)	6.39%	74.08% (+67.69%)

Table 7: CPU time and real time proportion when fuzzing on the latest released versions of GCC and LLVM ($n = 10^5$).

Tool	Step	CPU Time	Real Time
RVISmith	generation	0.021%	0.022%
gcc -O0	compilation	13.120%	33.700%
qemu	execution	13.317%	0.228%
gcc -O3	compilation	13.715%	33.219%
qemu	execution	12.994%	0.213%
clang -O0	compilation	11.665%	15.295%
qemu	execution	11.619%	0.233%
clang -O3	compilation	11.945%	16.858%
qemu	execution	11.605%	0.232%

```

for (size_t vl; avl > 0; avl -= vl){
    vl = __riscv_vsetvl_e64m8(avl);
    vint8m1_t mask_value = __riscv_vle8_v_i8m1(ptr_mask, vl);
    vbool8_t vmask = __riscv_vmseq_vx_i8m1_b8(mask_value, 1, vl);
    vuint8m1_t va = __riscv_vle8_v_u8m1(ptr_a, vl);
    vuint8m1_t vb = __riscv_vluxe_i8_v_u8m1(ptr_b, \
    __riscv_vsl_vx_u8m1(__riscv_vid_v_u8m1(vl), 0, vl), vl);
    vuint8m1_t vc = __riscv_vadd_vv_u8m1_m(vmask, va, vb, vl);
    __riscv_vse8_v_u8m1(ptr_c, vc, vl);
    /*some other intrinsics (ratio=8)*/
    ptr_mask += vl; ptr_a += vl; ptr_b += vl; ptr_c += vl;
}

```

Figure 8: [#117947] GCC at -O2/3 miscompiles this code. Pointers are updated by the CSR vlenb that is incorrectly configured by a vsetvli instruction. This bug results in half of the data being lost after computation.

The reason for this crash is that GCC tries to use a NULL pointer when parsing the vlmul_ext intrinsic. This bug has been fixed after we report it.

Illegal instruction. Figure 11 shows a program that triggers a miscompilation bug of LLVM. As previously discussed in this section, instructions that set the appropriate status of the frm CSR should be generated when compiling floating-point intrinsics. This LLVM bug generates a fsrmi instruction that tries to write an illegal value to the frm CSR, leading to a runtime crash. RVISmith and LLVM developers detect this bug nearly simultaneously, and this bug has been fixed.

We discuss the reason why these bugs cannot be detected by RIF. The bugs in Figure 8, Figure 9, and Figure 11 cannot be detected by

```

for (size_t vl; avl > 0; avl -= vl){
    vl = __riscv_vsetvl_e16m1(avl);
    vfloat16m1_t va = __riscv_vle16_v_f16m1(ptr_a, vl);
    va = __riscv_vfnmadd_vv_f16m1_rm(va, va, va, \
    __RISCV_FRM_RDN, vl);
    va = __riscv_vfmsac_vv_f16m1(va, va, va, vl);
    __riscv_vse16_v_f16m1(ptr_b, va, vl);
    ptr_a += vl; ptr_b += vl;
}

```

Figure 9: [#118103] GCC at -O3 miscompiles this code. The status of CSR frm is not restored between the first and second floating-point intrinsics, leading to an unintended rounding.

```

for (size_t vl; avl > 0; avl -= vl){
    vl = __riscv_vsetvl_e16m1(avl);
    vfloat16mf2_t va = __riscv_vle16_v_f16mf2(ptr_a, vl);
    vfloat16m1_t vb = __riscv_vlmul_ext_v_f16mf2_f16m1(va);
    ptr_a += vl;
}

```

Figure 10: [#117286] GCC at -O1/2/3/s crashes on this code. The LMUL extension intrinsic causes a segment fault in GCC.

```

for (size_t vl; avl > 0; avl -= vl){
    vl = __riscv_vsetvl_e32m1(avl);
    vfloat32m1_t va = __riscv_vlse32_v_f32m1(ptr_a, 4, vl);
    va = __riscv_vfsqrt_v_f32m1_rm(va, __RISCV_FRM_RNE, vl);
    va = __riscv_vfredosum_vs_f32m1_f32m1(va, va, vl);
    __riscv_vse32_v_f32m1(ptr_b, va, vl);
    ptr_a += vl; ptr_b += vl;
}

```

Figure 11: [#117909] Clang at -O0/1/2/3/s generates an illegal fsrmi instruction on this code.

RIF, as these bugs are triggered by complex combinations with multiple operation intrinsics and RIF does not support any combination of operation intrinsics. The bug in Figure 10 cannot be detected by RIF, because LMUL extension intrinsics and other related intrinsics that can trigger this bug are not supported by RIF.

5 Discussion

In this section, we present some limitations of RVISmith and multiple possible future directions.

Limitations. The main limitations of RVISmith are divided into three aspects. First, the generation of the invocation sequence of RVV intrinsics is purely random. A coverage-guided approach can

be developed in the future to enhance the coverage of specific intrinsics (e.g., segment load/store intrinsics) and combinations of intrinsics. Second, RVISmith focuses exclusively on combinations of RVV intrinsics within a single strip-mining loop, while RVV intrinsics in complex control flows remain untested. Previous work [22] has shown multiple compiler bugs related to loop optimizations. Third, RVISmith applies rule-based data generation for conditionally undefined intrinsics and indexed load/store intrinsics. To avoid undefined behaviors, RVISmith generates absolutely correct data for partial intrinsics instead of randomized data generation. This data generation results in certain scenarios remaining not covered.

From the perspective of security vulnerability detection, our work still overlooks three types of security vulnerabilities. First, vulnerabilities in emulators and hardware related to SIMD instructions are overlooked. While RVISmith is designed for fuzzing compilers for intrinsics and may help uncover such vulnerabilities, specialized fuzzing tools are still necessary. Second, vulnerabilities that are related to undefined behaviors are overlooked. RVISmith is intentionally designed to generate well-defined programs to avoid false positives, and no security-related code (to avoid undefined behaviors) is in the generated programs. However, some compiler-introduced security vulnerabilities arise from incorrect optimizations applied to security-related code [42]. Third, vulnerabilities in scenarios not covered by current RVISmith are overlooked, as discussed in the preceding limitations.

Future work. We propose three possible future directions derived from RVISmith. (1) Improvements of the approach for generating code with RVV intrinsics. Additional approaches could be developed in the future to address the aforementioned limitations of RVISmith to achieve higher coverage and detect more previously unknown bugs than RVISmith. (2) Fuzzing compilers for SIMD intrinsics in other ISAs. RVISmith is the first work in academia focused on fuzzing compilers for SIMD intrinsics to the best of our knowledge, and currently only RVV intrinsics are supported. Fuzzing compilers for other SIMD intrinsics, such as SSE/AVX for x86 [14] and Neon for ARM [2], still requires significant effort. Although the idea of RVISmith is general, the implementation of fuzzing tools on other ISAs requires solving domain-specific problems. (3) Fuzzing emulators and hardware for SIMD instructions. In the experiments of this paper, we use only the latest QEMU v9.1.0 to execute RISC-V ELF files after compilation, and we assume that there are no related bugs of QEMU. There are multiple emulators for RVV instructions in addition to QEMU, such as Spike [16], NEMU [41], and Berberis [11]. Future work could focus on improving the security and reliability of emulators and hardware for SIMD instructions.

6 Related Work

In this section, we discuss closely related work in compiler fuzzing.

Generation-based compiler fuzzing. Generation-based compiler fuzzing aims to detect compiler bugs by designing automatic program generators. The generated programs should conform to a specific syntax and typically be well defined. Csmith [43] is widely regarded as the most influential program generator that aims to detect C/C++ compiler bugs by differential testing. Complex heuristics are used in Csmith to avoid undefined behaviors, and hundreds

of compiler-optimization bugs have been detected upon the launch of Csmith. Multiple follow-on program generators have been developed after Csmith, each targeting specific features or programming languages. Morisset et al. [25] develop a tool based on Csmith to detect concurrency bugs in C/C++ programs. Lidbury et al. [20] propose CLSmith, a program generator for OpenCL compilers. Herklotz et al. [13] propose Verismith to generate Verilog programs for FPGA synthesis tools. Rustlantis [39] and RustSmith [31] are random program generators for fuzzing Rust compilers. YARPGen [21, 22] is a random program generator for fuzzing data-parallel programming languages; however, SIMD intrinsics are not supported by YARPGen. Numerous outstanding contributions have been made to generation-based compiler fuzzing, and the above-mentioned work represents only a portion of them.

Mutation-based compiler fuzzing. Another main approach to compiler fuzzing is mutation-based fuzzing, which generates programs by mutating seed programs (including real-world programs that are manually constructed and programs by existing generators). The most effective approaches in this type are the series of EMI (Equivalence Modulo Inputs) work [17, 18, 35]. The EMI work mutates seed programs by removing or modifying dead regions or inserting code into live regions, at the same time ensuring that the mutated program retains the same semantics. For C/C++ compiler fuzzing, GrayC [8] mutates seed programs under the guidance of code coverage; Creal [19] mutates seed programs by injecting real-world programs; and MetaMut [29] mutates seed programs with the help of large language models for compiler fuzzing. For JavaScript JIT compiler fuzzing, tools such as Fuzzilli [12], FuzzJIT [38], JIT-picking [4], and OptFuzz [37] implement mutation modules guided by domain-specific information.

7 Conclusion

In this paper, we have proposed an approach of fuzzing compilers for RVV intrinsics. We have implemented a fuzzer named RVISmith based on the ratified RVV intrinsic document in version 1.0. RVISmith has addressed the following challenges of fuzzing compilers for RVV intrinsics: (i) achieving high intrinsic coverage, (ii) improving sequence variety, and (iii) without known undefined behaviors. Experimental results have shown that RVISmith has achieved 11.5 times higher intrinsic coverage than the state-of-the-art fuzzer for RVV intrinsics. We have demonstrated the effectiveness of RVISmith by fuzzing three modern compilers for RVV intrinsics: GCC, LLVM, and XuanTie. RVISmith has detected 13 previously unknown bugs and these bugs have been reported to the corresponding compiler developers. Of these bugs, 10 have been confirmed and another 3 have been fixed by the compiler developers. We expect RVISmith to open up a new direction for detecting potential compiler bugs related to built-in functions, especially SIMD intrinsics.

Acknowledgments

This work was partially supported by Damo Academy (Hupan Laboratory) through Damo Academy (Hupan Laboratory) Innovative Research Program and National Natural Science Foundation of China under Grant No. 92464301. We would like to thank Ziyue Hua and Luyao Ren for discussing some technical details.

References

- [1] Alibaba DAMO Academy. 2024. XuanTie. <https://www.xrvm.com>.
- [2] ARM. 2024. Neon. <https://developer.arm.com/Architectures/Neon>.
- [3] Sara S. Baghsorkhi, Nalini Vasudevan, and Youfeng Wu. 2016. FlexVec: Auto-Vectorization for Irregular Loops. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2016)*. Association for Computing Machinery, New York, NY, USA, 697–710. doi:10.1145/2908080.2908111
- [4] Lukas Bernhard, Tobias Scharnowski, Moritz Schloegel, Tim Blazytko, and Thorsten Holz. 2022. JIT-Picking: Differential Fuzzing of JavaScript Engines. In *Proceedings of the 28th ACM SIGSAC Conference on Computer and Communications Security (CCS 2022)*. Association for Computing Machinery, New York, NY, USA, 351–364. doi:10.1145/3548606.3560624
- [5] Yishen Chen, Charith Mendis, Michael Carbin, and Saman Amarasinghe. 2021. VeGen: A Vectorizer Generator for SIMD and Beyond. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2021)*. Association for Computing Machinery, New York, NY, USA, 902–914. doi:10.1145/3445814.3446692
- [6] Clang. 2024. Undefined Behavior Sanitizer. <https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html>.
- [7] Vijay D'Silva, Mathias Payer, and Dawn Song. 2015. The Correctness-Security Gap in Compiler Optimization. In *Proceedings of the 36th IEEE Security and Privacy Workshops (SPW 2015)*. IEEE Computer Society, USA, 73–87. doi:10.1109/SPW.2015.33
- [8] Karine Even-Mendoza, Arindam Sharma, Alastair F. Donaldson, and Cristian Cadar. 2023. GrayC: Greybox Fuzzing of Compilers and Analysers for C. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1219–1231. doi:10.1145/3597926.3598130
- [9] Jing Ge Feng, Ye Ping He, Qiu Ming Tao, and Fazli Wahid. 2021. Evaluation of Compilers' Capability of Automatic Vectorization Based on Source Code Analysis. *Scientific Programming* 2021, 1 (2021), 3264624. doi:10.1155/2021/3264624
- [10] GCC. 2024. The GNU Compiler Collection. <https://gcc.gnu.org>.
- [11] Google. 2024. Berberis. https://android.googlesource.com/platform/frameworks/libs/binary_translation/+refs/heads/main/README.md.
- [12] Samuel Groß, Simon Koch, Lukas Bernhard, Thorsten Holz, and Martin Johns. 2023. FUZZILLI: Fuzzing for JavaScript JIT Compiler Vulnerabilities. In *Proceedings of 30th edition of the Network and Distributed System Security Symposium (NDSS 2023)*. Internet Society, USA. doi:10.14722/ndss.2023.24290
- [13] Yann Herklotz and John Wickerson. 2020. Finding and Understanding Bugs in FPGA Synthesis Tools. In *Proceedings of the 28th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA 2020)*. Association for Computing Machinery, New York, NY, USA, 277–287. doi:10.1145/3373087.3375310
- [14] Intel. 2024. Intel® Intrinsics Guide. <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>.
- [15] RISC-V International. 2024. RISC-V Vector Intrinsic Document. <https://github.com/riscv-non-isa/rvv-intrinsic-doc>.
- [16] RISC-V International. 2024. Spike. <https://github.com/riscv-software-src/riscv-isa-sim>.
- [17] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler Validation via Equivalence Modulo Inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2014)*. Association for Computing Machinery, New York, NY, USA, 216–226. doi:10.1145/2594291.2594334
- [18] Vu Le, Chengnian Sun, and Zhendong Su. 2015. Finding Deep Compiler Bugs via Guided Stochastic Program Mutation. In *Proceedings of the 30th ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2015)*. Association for Computing Machinery, New York, NY, USA, 386–399. doi:10.1145/2814270.2814319
- [19] Shaohua Li, Theodoros Theodoridis, and Zhendong Su. 2024. Boosting Compiler Testing by Injecting Real-World Code. In *Proceedings of the 45th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2024)*. Association for Computing Machinery, New York, NY, USA, 223–245. doi:10.1145/3656386
- [20] Christopher Lidbury, Andrei Lascu, Nathan Chong, and Alastair F. Donaldson. 2015. Many-Core Compiler Fuzzing. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2015)*. Association for Computing Machinery, New York, NY, USA, 65–76. doi:10.1145/2737924.2737986
- [21] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2020. Random Testing for C and C++ Compilers with YARPGen. In *Proceedings of the 35th ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2020)*. Association for Computing Machinery, New York, NY, USA, 1–25. doi:10.1145/3428264
- [22] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2023. Fuzzing Loop Optimizations in Compilers for C++ and Data-Parallel Languages. In *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2023)*. Association for Computing Machinery, New York, NY, USA, 1826–1847. doi:10.1145/3591295
- [23] LLVM. 2024. The LLVM Compiler Infrastructure. <https://llvm.org>.
- [24] Charith Mendis, Cambridge Yang, Yewen Pu, Dr.Saman Amarasinghe, and Michael Carbin. 2019. Compiler Auto-Vectorization with Imitation Learning. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NeurIPS 2019)*. Curran Associates Inc., Red Hook, NY, USA. <https://dl.acm.org/doi/10.5555/3454287.3455597>
- [25] Robin Morisset, Pankaj Pawan, and Francesco Zappa Nardelli. 2013. Compiler Testing via a Theory of Sound Optimisations in the C11/C++11 Memory Model. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2013)*. Association for Computing Machinery, New York, NY, USA, 187–196. doi:10.1145/2491956.2491967
- [26] Dorit Nuzman and Richard Henderson. 2006. Multi-Platform Auto-Vectorization. In *Proceedings of the 4th International Symposium on Code Generation and Optimization (CGO 2006)*. IEEE Computer Society, USA, 281–294. doi:10.1109/CGO.2006.25
- [27] Dorit Nuzman, Ira Rosen, and Ayal Zaks. 2006. Auto-Vectorization of Interleaved Data for SIMD. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2006)*. Association for Computing Machinery, New York, NY, USA, 132–143. doi:10.1145/1133981.1133997
- [28] OpenCV.AI. 2024. OpenCV. <https://opencv.org/>.
- [29] Xianfei Ou, Cong Li, Yanyan Jiang, and Chang Xu. 2024. The Mutators Reloaded: Fuzzing Compilers with Large Language Model Generated Mutation Operators. In *Proceedings of 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2024)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3622781.3674171
- [30] Angela Pohl, Biagio Cosenza, Mauricio Alvarez Mesa, Chi Ching Chi, and Ben Juurlink. 2016. An Evaluation of Current SIMD Programming Models for C++. In *Proceedings of the 3rd Workshop on Programming Models for SIMD/Vector Processing (WPMVP 2016)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/2870650.2870653
- [31] Mayank Sharma, Pingshi Yu, and Alastair F. Donaldson. 2023. RustSmith: Random Differential Compiler Testing for Rust. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1483–1486. doi:10.1145/3597926.3604919
- [32] SiFive. 2024. RVV Intrinsic Fuzzing (RIF). <https://github.com/sifive/riscv-vector-intrinsic-fuzzing>.
- [33] Sergi Siso, Wes Armour, and Jeyarajan Thiayagalingam. 2019. Evaluating Auto-Vectorizing Compilers through Objective Withdrawal of Useful Information. *ACM Transactions on Architecture and Code Optimization (TACO)* 16, 4 (2019), 1–23. doi:10.1145/3356842
- [34] Alen Stojanov, Ivaylo Toskov, Tiark Rompf, and Markus Püschel. 2018. SIMD Intrinsics on Managed Language Runtimes. In *Proceedings of the 16th International Symposium on Code Generation and Optimization (CGO 2018)*. Association for Computing Machinery, New York, NY, USA, 2–15. doi:10.1145/3168810
- [35] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding Compiler Bugs via Live Code Mutation. In *Proceedings of the 31st ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2016)*. Association for Computing Machinery, New York, NY, USA, 849–863. doi:10.1145/2983990.2984038
- [36] Theodoros Theodoridis and Zhendong Su. 2024. Refined Input, Degraded Output: The Counterintuitive World of Compiler Behavior. In *Proceedings of the 45th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2024)*. Association for Computing Machinery, New York, NY, USA, 671–691. doi:10.1145/3656404
- [37] Jiming Wang, Yan Kang, Chenggang Wu, Yuhao Hu, Yue Sun, Jikai Ren, Yuanming Lai, Mengyao Xie, Charles Zhang, Tao Li, and Zhe Wang. 2024. OptFuzz: Optimization Path Guided Fuzzing for JavaScript JIT Compilers. In *Proceedings of the 33rd USENIX Conference on Security Symposium (USENIX Security 2024)*. USENIX Association, Philadelphia, PA, 865–882. <https://www.usenix.org/conference/usenixsecurity24/presentation/wang-jiming>
- [38] Junjie Wang, Zhiyi Zhang, Shuang Liu, Xiaoning Du, and Junjie Chen. 2023. FuzzJIT: Oracle-Enhanced Fuzzing for JavaScript Engine JIT Compiler. In *Proceedings of the 32nd USENIX Conference on Security Symposium (USENIX Security 2023)*. USENIX Association, Anaheim, CA, 1865–1882. <https://www.usenix.org/conference/usenixsecurity23/presentation/wang-junjie>
- [39] Qian Wang and Ralf Jung. 2024. Rustlantis: Randomized Differential Testing of the Rust Compiler. In *Proceedings of the 39th ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2024)*. Association for Computing Machinery, New York, NY, USA, 1955–1981. doi:10.1145/3689780
- [40] Xi Wang, Nikolai Zeldovich, M Frans Kaashoek, and Armando Solar-Lezama. 2015. A Differential Approach to Undefined Behavior Detection. *ACM Transactions on Computer Systems (TOCS)* 33, 1 (2015), 1–29. doi:10.1145/2699678
- [41] XiangShan. 2024. NEMU. <https://github.com/OpenXiangShan/NEMU>.
- [42] Jianhao Xu, Kangjie Lu, Zhengjie Du, Zhu Ding, Linke Li, Qiushi Wu, Mathias Payer, and Bing Mao. 2023. Silent Bugs Matter: A Study of Compiler-Introduced

Security Bugs. In *Proceedings of the 32nd USENIX Conference on Security Symposium (USENIX Security 2023)*. USENIX Association, Anaheim, CA, 3655–3672. <https://www.usenix.org/conference/usenixsecurity23/presentation/xu-jianhao>

- [43] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2011)*. Association for Computing Machinery, New York, NY, USA, 283–294. doi:10.1145/1993498.1993532